

There are 6 documents in the
submission – numbered from 1 to 6.

MECH0107 COURSEWORK 2

Multi-Objective Optimisation of an Aircraft Maintenance Panel

Contents

1 – Introduction	1
2 – Methodology	1
a) Data pipeline	1
b) GPR model	2
c) Neural Network Model	4
d) NSGA-II	7
3 – Results	8
a) Validation accuracy	8
b) Pareto model	9
c) Surrogate validation against FEA	10
4 – Discussion	10
a) Computational Efficiency	10
b) Model selection	11
c) Conditions that favour NN models	11
d) Revised constraints	11
e) Design improvements	12
5 – Conclusion	12
6 – Appendix	13
7 – Acknowledgement and references	13

1 – Introduction

Aircraft structures are designed to achieve a precise balance between structural integrity and minimal weight. Maintenance panels provide essential access to the internal systems of the wing for inspection and servicing, however the geometric discontinuity introduced by their port opening generates stress concentrations in the surrounding structure. Optimising the shape of these panels is therefore essential, as reducing their mass improves aircraft performance, lowers fuel consumption and associated carbon emissions, while minimising stress concentrations enhances structural safety and extends service life.

Traditional design optimisation for these panels relies on repeated Finite Element Analysis (FEA) simulations to evaluate candidate designs, which becomes computationally expensive when a large design space must be explored. Each FEA evaluation of the maintenance panel, defined by the four design variables seen in Figure 1, requires a full simulation run taking tens of seconds, meaning a multi-objective optimisation algorithm running directly on FEA would demand many hours of computing time, making it impractical for iterative design work. Using surrogate models instead offers an efficient solution to this problem. By training a data-driven model on a defined size set of FEA results, it becomes possible to approximate the relationship between design variables and structural performance, specifically the maximum von Mises stress σ_{max} , at a fraction of the computational cost [1]. Once trained, a surrogate can evaluate thousands of candidate designs in seconds, enabling rapid exploration of the Pareto-optimal trade-off between minimising internal panel stress σ_{max} and panel mass m [2].

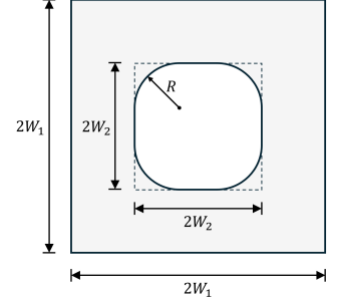


Figure 1: Geometry of the maintenance plate

This report presents the development and evaluation of two surrogate models: a Gaussian Process Regression (GPR) model and a Neural Network (NN) model. Both are trained on FEA data generated using Abaqus, with design variables sampled across bounds defined by functional constraints arising from neighbouring panel proximity and maintenance access requirements. Each surrogate is then integrated into a multi-objective optimisation algorithm to produce a Pareto front, from which optimal panel designs can be selected. The report details the data sampling strategy, preprocessing steps, and model design choices for each approach, before presenting error evaluations, Pareto front results, and a comparative discussion of the suitability of each method for this application.

2 – Methodology

a) Data pipeline

The maintenance panel is modelled in Abaqus as a thin square plate under biaxial loading, with a fixed stress ratio of 2. Symmetry is used by modelling only one quarter of the plate, reducing computational cost while preserving accuracy. The material is aluminium alloy with Young's modulus $E = 70$ GPa, Poisson's ratio $\nu = 0.33$, and density $\rho = 2700$ kg/m³. The four design variables governing the panel geometry, namely the plate half-width W_1 , port half-width W_2 , fillet radius R , and plate thickness t , are varied within the bounds defined in Table 1:

Variable	Lower bound (m)	Upper bound (m)
W_1	0.3	0.7
W_2	0.1	0.15
R	0.03	0.07
t	0.01	0.02

Table 1: Design variable bounds for the maintenance panel shape optimisation

The data acquisition process was automated using a MATLAB script which interfaces directly with Abaqus to generate the training dataset used for both surrogate models. For each sample, the script writes the design variable values to an input text file, launches an Abaqus-Python script that updates the model geometry, runs the biaxial simulation, and extracts the maximum von Mises stress and simulation runtime from the output file. The mass is computed analytically within the loop using the following equation:

$$m(W_1, W_2, R, t) = \rho t [4W_1^2 - 4W_2^2 + (4 - \pi)R^2] \quad (1)$$

Each result is then appended to a .csv file upon completion, containing the sample number, design parameters, maximum von Mises stress, and Abaqus simulation runtime.

To populate the design space, Latin Hypercube Sampling (LHS) was used to generate the input design variable combinations. Unlike random sampling, LHS enforces a stratified structure across each variable dimension, ensuring uniform coverage of the full design space [3]. The maximin criterion was applied with 100 iterations to maximise the minimum distance between sample points, reducing clustering and improving space-filling properties across the four-dimensional input space.

A total of 300 samples were generated over a total wall-clock acquisition time of 8 177.35 seconds, at an average of 27.3 seconds per Abaqus simulation. This large dataset size provides sufficient flexibility to explore multiple training set sizes for both surrogate models, and the per-sample runtime serves as the baseline for the computational efficiency analysis in the methodology.

b) GPR model

The Gaussian Process Regression (GPR) is a non-parametric, probabilistic method that models input-output relationships by placing a prior distribution over functions and updating it with observed data. It is well-suited for surrogate modelling on small datasets with nonlinear responses, providing both a mean prediction and an uncertainty estimate at each point in the design space [4].

The training dataset is read from the .csv file, with the four design variables defined as inputs and the maximum von Mises stress as the output. The dataset is partitioned into a training set and a validation set using an 80/20 split [5] with prior shuffling to avoid ordering bias, providing a sufficiently large training set to capture the nonlinear stress response while retaining enough validation points for a statistically meaningful accuracy evaluation. Before training, inputs are normalised using a MinMax scaler, mapping the variables to the range [0, 1], as the design variables span different physical magnitudes: W_1 ranges over 0.4 m while t over 0.01 m. Without normalisation, the kernel length-scale hyperparameters would be dominated by the larger-magnitude variables, leading to a poorly calibrated kernel that underweights the influence of smaller-scale variables such as t and R . The output σ_{max} is standardised to zero mean and unit variance using a Standard scaler, improving the numerical conditioning of the marginal likelihood optimisation [4].

The GPR is constructed using a composite kernel consisting of a constant scaling term C multiplied by a base kernel, with the RBF and Matérn families considered alongside an additive White Noise kernel [4]. The model is trained by maximising the log marginal likelihood with respect to all kernel hyperparameters, automatically balancing model fit against complexity. To reduce the risk of converging to a local optimum, the hyperparameter optimiser is restarted from multiple random initialisations, with the number of restarts investigated systematically below.

Two accuracy metrics are used to judge the accuracy of the model alongside its runtime. RMSE measures the average magnitude of prediction errors in the same units as the output variable (MPa):

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\sigma_i - \hat{\sigma}_i)^2} \quad (2)$$

With σ_i the FEA stress value, $\hat{\sigma}_i$ the surrogate value, and n the number of samples. Its quadratic nature penalises large prediction errors more heavily, which is particularly relevant for a structural safety application where significant stress underestimation carries direct safety consequences. The other metric is R^2 , which measures the proportion of variance in the true stress values explained by the surrogate, with a value of 1 indicating a perfect fit. Finally, the run time is assessed for each iteration. The runtime is not repeatable and varies from one run to the other. All optimisation steps are conducted sequentially, with each variable fixed upon selection before the next is varied. To balance predictive accuracy and computational efficiency, the optimal configuration at each stage is selected as the shortest training time within 15% of the lowest RMSE. It is used as the sole selection metric as it is expressed in physically interpretable units (MPa) and penalises large errors more heavily through its quadratic nature, making it directly meaningful for a structural safety application. Differences within the 15%

band are negligible relative to stress values in the tens of MPa, making computational efficiency the appropriate tiebreaker. This criterion is applied consistently across all experiments, enabling the full pipeline to be automated, reproducible, and timed end-to-end. Initial GPR parameters are an RBF kernel with 5 restarts. Selected values are highlighted in green, with the baseline indicated in italics.

1. Training Sample Size

Training set size was investigated first, as it determines the information available to the model:

Sample size	RMSE (MPa)	R^2	Training GPR (s)
50	1.4466	0.9873	0.25
100	0.4435	0.9976	0.55
150	0.4525	0.9980	0.79
200	0.8304	0.9961	1.10
250	0.8991	0.9953	2.42
300	0.4236	0.9987	3.28

Table 2: Effect of training sample size on GPR validation and training time

As shown in Table 2, RMSE drops sharply by approximately 69% between 50 and 100 samples, indicating the model is data-starved at smaller sizes. Beyond 150 samples, accuracy becomes inconsistent, with RMSE increasing by 84% at 200 samples before recovering at 300 samples, suggesting variability reflects favourable random splits rather than genuine improvement. The 15% tolerance criterion is not applied here, as sample size directly determines the information available to all subsequent experiments and selecting too few samples (ex: 100) would systematically bias downstream results. A training size of 300 samples was therefore selected on accuracy grounds. Although the increase from 150 to 300 samples yields only a 6% RMSE reduction, the absolute training time of 3.28 s remains negligible in terms of model training, with the dominant cost being the FEA data acquisition rather than the GPR training itself.

2. Kernel Type

The kernel defines how similarity between input designs translates into similarity in predicted stress. Six configurations were evaluated. All include a constant scaling term C controlling the overall amplitude of the predicted stress response. The RBF kernel is defined as:

$$k(x, x') = C * \exp\left(-\frac{\|x - x'\|^2}{2l^2}\right) \quad (3)$$

where l is the length-scale parameter controlling the rate of correlation decay with distance. It assumes an infinitely differentiable function, appropriate for this structural problem where small geometric changes produce gradual variations in stress. [6]

The other choice is the Matérn kernel that generalises the RBF by introducing a smoothness parameter ν controlling function differentiability. Three values were evaluated: $\nu = 0.5$, assuming only continuity; 1.5 assuming once-differentiable functions; and 2.5 assuming twice-differentiable functions, to assess whether a rougher response surface better captures local stress variations around the fillet radius or port edges [4]. An additive White Noise kernel was also evaluated, introducing a small noise term along the covariance matrix diagonal to absorb numerical irregularities in the FEA training data, improving numerical stability and preventing overfitting [7].

Kernel type	RMSE (MPa)	R^2	Training GPR (s)
<i>C * RBF (baseline)</i>	0.4236	0.9980	3.28
C * Matérn 0.5	0.5936	0.9974	5.53
C * Matérn 1.5	0.1851	0.9997	3.62
C * Matérn 2.5	0.1501	0.9998	5.40
C * Matérn 2.5 + WK	0.1295	0.9999	8.03
C * RBF + WK	0.1373	0.9999	2.92

Table 3: Effect of kernel configuration on GPR validation RMSE and training time

As shown in Table 3, accuracy improves consistently with increasing ν across the Matérn family, with the RBF kernel confirming this trend by outperforming all base Matérn configurations as it assumes $\nu \rightarrow \infty$. The White Noise kernel proves beneficial as adding it to Matérn 2.5 reduces RMSE by approximately 14%, while adding it to RBF reduces RMSE by approximately 70%, confirming its role in absorbing FEA numerical irregularities and improving optimisation stability. Applying the 15% tolerance criterion, both RBF + WK and Matérn 2.5 + WK fall within the acceptable RMSE band, but RBF + WK was selected as it achieves comparable accuracy at a training time approximately 64% shorter than the other solution.

3. Number of Optimiser Restarts

This parameter controls how many times the hyperparameter optimisation is reinitialised from different starting points, reducing the risk of converging to a local optimum in the marginal likelihood landscape. Increasing restarts improves the likelihood of finding the global optimum at the cost of proportionally longer training time:

Restarts	RMSE (MPa)	R^2	Training GPR (s)
0	11.8062	-0.0198	0.17
2	11.8062	-0.0198	0.84
3	0.1373	0.9999	2.91
4	0.1373	0.9999	2.97
5 (baseline)	0.1373	0.9999	2.92
10	0.1373	0.9999	9.91

Table 4: Effect of number of optimiser restarts on GPR validation and training time

As shown in Table 4, results at 0 and 2 restarts produce negative R^2 values, confirming the optimiser has failed to converge to a meaningful solution. Convergence is first observed at 3 restarts, yielding the optimal RMSE and R^2 . Beyond 3 restarts, accuracy remains unchanged while training time increases. A restart value of 3 was therefore selected.

4. Final GPR Configuration

The optimal GPR configuration was established sequentially: 300 training samples were selected as the point beyond which accuracy gains become inconsistent, the C * RBF + WhiteKernel was identified as the optimal kernel with the noise term proving essential for numerical stability, and 3 restarts were determined as the minimum required for consistent convergence. The final model achieves a validation RMSE of 0.1373 MPa and R^2 of 0.9999.

c) Neural Network Model

A feedforward Neural Network (NN) was developed as a second surrogate. Unlike GPR, a neural network is a parametric model learning a hierarchical input-output representation through successive weighted transformations, making it more flexible but more sensitive to architectural and training choices.

The same original dataset is used, with a 70/15/15 train/validation/test split rather than the 80/20 used for GPR. The validation set is used for early stopping and hyperparameter selection, while the test set is held out entirely to provide an unbiased accuracy estimate. Using the validation set for both purposes would produce an optimistic accuracy estimate, as the model has indirectly been exposed to those points during hyperparameter selection [8]. The dataset is first split into 70% training and 30% temporary, the latter divided equally into validation and test subsets. Inputs are normalised to $[0,1]$ and σ_{max} standardised to zero mean and unit variance. Without normalisation, larger variables such as W_1 would dominate weight updates, leading to unstable model convergence. During hyperparameter search, each configuration is evaluated under a single fixed seed to maintain computational efficiency across the large number of experiments conducted. Once all hyperparameters are finalised, the optimal configuration is retrained five times across fixed seeds (1 to 5) to ensure reported metrics reflect consistent model behaviour rather than a single favourable initialisation, with the best-performing run identified by lowest test set RMSE retained for optimisation. Early stopping is applied throughout to prevent overfitting [9]. Validation loss is monitored at each epoch, where one epoch represents one complete pass of the training dataset through the network, with training terminated if no improvement is observed for the specified patience and best weights restored. A maximum of 1000 epochs is permitted, with the Adam optimiser and mean squared error loss function used throughout. Throughout the optimisation steps, the same 15% tolerance criterion is applied throughout.

1. Training sample size

Training sample size was investigated first, with all other hyperparameters held at default values: one hidden layer, 16 neurons, ReLU activation, learning rate of 0.001, batch size of 16, and patience of 25 epochs.

Sample size	RMSE (MPa)	R^2	Training + validation NN (s)
60	2.0596	0.9585	25.19
100	1.7680	0.9717	18.82
160	0.8972	0.9905	45.25
200	1.5019	0.9888	32.51
260	1.8566	0.9825	19.97
300	1.1098	0.9912	18.16

Table 5: Effect of training sample size on NN validation and training time

As shown in Table 5, RMSE decreases broadly with increasing dataset size, dropping by approximately 57% between 60 and 160 samples, though non-monotonic increases beyond 160 samples reflect the influence of the fixed random split rather than a systematic trend. Applying the selection criterion, 160 samples achieves the lowest RMSE and highest R^2 across all tested configurations, with no other size falling within the 15% tolerance band. It was therefore selected.

2. Network architecture (layers and neurons)

The number of hidden layers and neurons per layer determines the model capacity to learn the nonlinear stress response: too shallow risks underfitting, while too deep risks overfitting and poor generalisation. All other hyperparameters were held at default values.

Architecture	RMSE (MPa)	R^2	Training + validation NN (s)
8	2.1505	0.9456	38.90
2 x 8	2.1180	0.9472	17.86
16 (baseline)	0.8972	0.9905	58.43
2 x 16	1.3167	0.9796	25.28
32	0.6677	0.9948	74.87
2 x 32	0.7341	0.9937	46.15
64	1.1647	0.9840	26.97
2 x 64	0.6578	0.9949	39.29

Table 6: Effect of network architecture on NN validation and training time

As shown in Table 6, no consistent trend emerges across single-layer architectures, with RMSE decreasing up to 32 neurons before increasing at 64. Among two-layer architectures, 64×64 achieves the lowest RMSE and highest R^2 , confirming that added depth improves the network's ability to learn the nonlinear stress response more effectively than simply increasing single-layer width. Applying the selection criterion, 32 and 64×64 both fall within the 15% tolerance band of the lowest RMSE, with 64×64 selected as it achieves superior accuracy at a meaningfully shorter training time than the single 32-neuron configuration.

3. Activation function

Without nonlinear activations, the network reduces to a linear model regardless of depth. Four functions were evaluated: ReLU, computationally efficient but susceptible to dying neurons for negative inputs; Leaky ReLU, which addresses this with a small negative gradient; tanh, symmetric about zero and mapping outputs to $[-1, 1]$, aiding gradient flow in deeper networks; and sigmoid, mapping outputs to $[0, 1]$ and typically used for classification but evaluated here for completeness.

Activation function	RMSE (MPa)	R^2	Training + validation NN (s)
ReLU (baseline)	0.6578	0.9949	39.29
Tanh	0.2659	0.9992	74.22
Sigmoid	1.6372	0.9684	97.10
Leaky_Relu	0.7281	0.9938	35.17

Table 7: Effect of activation function on NN training and validation metrics

As shown in Table 7, sigmoid performs poorly, confirming it is unsuitable for this regression task. Leaky ReLU achieves lower RMSE than the ReLU baseline, while tanh achieves the lowest RMSE and highest R^2 of all tested activations by a considerable margin. Although tanh incurs a training time approximately twice that of Leaky ReLU, the accuracy improvement justifies this cost given that training is a one-time expense. Tanh was therefore selected as the optimal activation function, with remaining hyperparameter experiments offering potential for further RMSE improvement.

4. Learning rate

The learning rate controls the Adam optimiser step size during weight updates: too high risks overshooting the loss minimum, while too low risks slow convergence or entrapment in a local minimum. Six values were evaluated spanning two orders of magnitude.

Learning rate	RMSE (MPa)	R^2	Training + validation NN (s)
0.05	0.3513	0.9985	7.53
0.01	0.2943	0.9990	16.84
0.005	0.4093	0.9980	13.38
0.001 (<i>baseline</i>)	0.2659	0.9992	74.22
0.0005	0.6568	0.9981	54.01
0.0001	0.4061	0.9720	74.02

Table 8: Effect of learning Rate on NN training and validation metrics

As shown in Table 8, a learning rate of 0.05 confirms the step size is too large, with the optimiser overshooting the loss minimum. Performance improves at 0.01 and continues gradually with smaller values, however the marginal RMSE gains below 0.01 come at a training time increase of approximately 4-fold at 0.001 and 3-fold at 0.0005, for a reduction negligible in the context of stress values in the tens of MPa. A learning rate of 0.01 achieves the most adequate RMSE and R^2 variance for its training time across all options. Given the NN training time is already significantly higher than GPR, a learning rate of 0.01 was selected as the optimal balance between accuracy, stability, and computational efficiency.

5. Batch size

Batch size defines the number of samples processed per weight update, controlling the trade-off between gradient estimate accuracy and computational efficiency. Smaller batches produce noisier but more frequent updates, while larger batches produce smoother estimates with fewer updates per epoch. Three values were evaluated:

Batch size	RMSE (MPa)	R^2	Training + validation NN (s)
16 (<i>baseline</i>)	0.2943	0.9990	16.84
32	1.1982	0.9831	13.620
64	0.4470	0.9976	34.522

Table 9: Effect of batch size on NN Training and validation metrics

As shown in Table 9, batch size 16 achieves the lowest RMSE and highest R^2 . Batch size 32 produces the highest RMSE and lowest R^2 of all tested configurations, confirming it is the least accurate option here. Applying the selection criterion, batch size 16 is selected as it achieves the lowest RMSE at a competitive training time, with no configuration within the tolerance band offering a meaningful efficiency advantage over it.

6. Early stopping

For Early stopping, too low a value risks premature termination before convergence, while too high a value allows unnecessary epochs beyond the optimal weights. Three values were evaluated:

Patience	RMSE (MPa)	R^2	Training + validation NN (s)
10	1.9968	0.9531	4.78
25 (<i>baseline</i>)	0.2943	0.9990	25.85
50	0.2206	0.9994	35.901

Table 10: Effect of early stopping patience setting on NN Training and validation metrics

As shown in Table 10, patience 10 confirms premature termination, with RMSE substantially higher than both other configurations. Increasing patience consistently improves accuracy, with patience 50 achieving the lowest RMSE and highest R^2 . Patience 25 falls outside the 15% tolerance band of the lowest RMSE, confirming the accuracy difference is physically meaningful. Patience 50 was therefore selected as it ensures full convergence without redundant computation beyond the optimal weights.

Final model configuration

The final neural network configuration uses 160 training samples with a 70/15/15 split, a 64×64 architecture with tanh activation, a learning rate of 0.01, batch size of 16, and early stopping patience of 50 within a maximum of 1000 epochs. Since neural network training is sensitive to weight initialisation, the final configuration is trained across five fixed seeds, with the best-performing run identified by lowest test set RMSE retained for optimisation:

Seed	RMSE (MPa)	R^2	Training + validation NN (s)
1(baseline)	0.2206	0.9994	35.90
2	0.3546	0.9973	28.52
3	0.2206	0.9995	29.30
4	0.4374	0.9989	27.33
5	0.9200	0.9907	15.10

Table 11: Effect of fixed seeds on NN Training and validation metrics

As shown in Table 11, RMSE varies substantially across seeds, ranging from 0.2206 to 0.9200 MPa with a standard deviation of 0.2736 MPa, confirming the seed-dependent variability inherent to neural network training. R^2 similarly ranges from 0.9907 to 0.9995, with training times varying by over 20 seconds across runs, indicating that different initialisations converge at different epochs. Seeds 1 and 3 achieve the joint lowest RMSE of 0.2206 MPa, with seed 3 selected as the best-performing run on the basis of shortest training time. This non-reproducibility further reinforces the stability advantage of GPR, which produces identical results regardless of initialisation.

d) NSGA-II

The Non-dominated Sorting Genetic Algorithm II (NSGA-II) was used for multi-objective shape optimisation, evolving candidate designs across successive generations through selection, crossover, and mutation. Solutions are ranked by Pareto dominance at each generation, with a crowding distance metric preserving front diversity. NSGA-II is well-suited here as it handles two competing objectives without scalarisation and produces a full Pareto front in a single run. The two objectives are simultaneous minimisation of σ_{max} and panel mass m . Mass is computed analytically using Equation 1, while σ_{max} is evaluated by the trained surrogate in place of a direct FEA call. Design variable bounds from Table 1 are enforced as hard constraints throughout.

The surrogate is integrated by replacing the FEA call within the objective evaluation function. At each generation, candidate design variables are scaled using the fitted MinMax scaler, passed to the surrogate for stress prediction, and inverse-transformed to recover the von Mises stress in MPa. Population size and number of generations were investigated systematically for both surrogates to identify the configuration producing a well-distributed Pareto front at minimal cost. Front quality was assessed using mean spacing between neighbouring Pareto points and its standard deviation, where a low mean spacing combined with low standard deviation confirms a dense and evenly distributed front. Optimisation runtime was recorded alongside these metrics.

NSGA-II Settings: GPR Surrogate

Several combinations of population size and number of generations were evaluated. The reported runtime corresponds exclusively to the NSGA-II process, excluding model training time reported previously.

Population size	Generations	Mean Spacing	Spacing Standard Deviation	Runtime (s)
50	50	2.5195	1.4536	0.374
100	50	1.5040	1.0158	0.664
100	100	1.5645	0.9144	1.258
150	100	1.0670	0.5995	1.612
150	150	1.0836	0.6634	2.764

Table 12: Effect of population size and number of generations on the GPR Pareto front

As shown in Table 12, increasing population size from 100 to 150 produces a meaningful reduction in both mean spacing and standard deviation, indicating a denser and more uniformly distributed front. Increasing generations beyond 100 yields degrades the mean spacing, suggesting additional generations introduce redundant solutions rather than improving front distribution, at a runtime increase of approximately 70%. A population size of 150 and 100 generations for the GPR model was therefore selected as the optimal balance between front quality and computational efficiency.

NSGA-II Settings: NN Surrogate

Population size	Generations	Mean Spacing	Spacing Standard Deviation	Runtime (s)
50	50	3.0817	2.9889	3.505
100	50	1.4924	1.2259	3.439
100	100	1.5256	0.8779	6.628
150	100	0.9923	0.6247	7.087
150	150	1.0056	0.5550	10.792

Table 13: Effect of population size and number of generations on the NN Pareto front

As shown in Table 13, the same trend observed for GPR is apparent for the NN surrogate: increasing population size from 100 to 150 consistently improves both spacing metrics, while increasing generations beyond 100 yields no meaningful improvement. A population of 150 and 100 generations was therefore selected for the NN optimisation, maintaining consistency with the GPR settings to ensure a fair comparison between the two Pareto fronts.

3 – Results

This section presents the predictive accuracy and optimisation outputs of both surrogate models using the configurations established in Section 2. Predictive performance is first evaluated on held-out data before each surrogate is used within NSGA-II to generate a Pareto front. Selected Pareto designs are then validated directly in Abaqus to confirm physical accuracy.

a) Validation accuracy

Predictive accuracy is evaluated by comparing surrogate predictions against true FEA values on held-out data. The GPR is evaluated on 60 validation points from the 300-sample dataset, while the NN is evaluated on the 24-point test set from the 160-sample dataset, withheld entirely throughout hyperparameter selection. The best-performing NN run across the five seeds is used. Predicted versus actual scatter plots are presented for both models alongside RMSE and R^2 metrics, with perfect accuracy corresponding to all points lying on the ideal diagonal. The training and Pareto runtime represents the time required to train the surrogate with its finalised hyperparameters and generate the Pareto front. The total optimisation runtime represents the cumulative time across all hyperparameter experiments conducted during model selection, including every configuration evaluated at each stage.

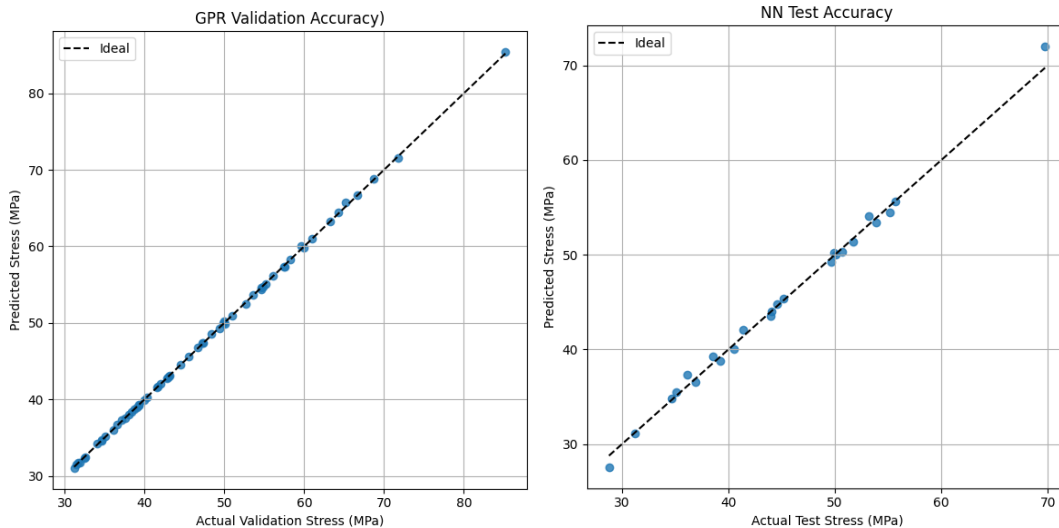


Figure 2, 3: Predicted versus actual maximum von Mises stress for the GPR and NN surrogate

GPR Metrics	Value
Validation RMSE (MPa)	0.1373
Validation R^2	0.9999
Training + Pareto runtime (s)	4.26
Total optimisation runtime (s)	60.90

NN Metrics	Value
Test RMSE (MPa)	0.2206
Test R^2	0.9995
Training + Pareto runtime (s)	46.31
Total optimisation runtime (s)	1404.40

Table 14, 15: GPR and NN validation and training metrics

As shown in Figures 2 and 3, GPR predictions cluster tightly and uniformly along the ideal diagonal across the full stress range. The NN shows a similar trend but with greater scatter, particularly in the lower stress range, and less uniform distribution of solutions. The GPR achieves an RMSE approximately 38% lower and R^2 closer to unity, while completing training and Pareto front generation approximately ten times faster than the NN surrogate.

b) Pareto model

The trained surrogates were integrated into NSGA-II to generate a Pareto front for each model. Three designs are selected from each front: lowest stress, lowest mass, and an optimal compromise identified as the point minimising the Euclidean distance to the ideal origin in normalised objective space.

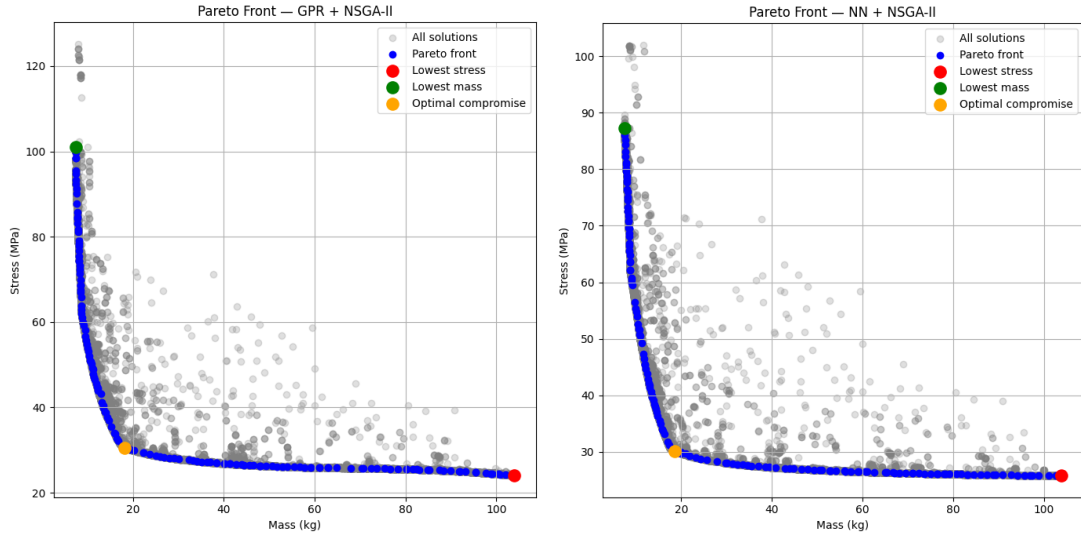


Figure 4,5: Pareto fronts obtained from shape optimisation using the GPR and NN surrogate models

Solution	Model	W_1 (m)	W_2 (m)	R (m)	t (m)	Mass (kg)	VM stress (MPa)
Lowest stress	GPR	0.700	0.100	0.070	0.020	103.90	24.01
	NN	0.700	0.100	0.070	0.020	103.87	25.75
Lowest mass	GPR	0.300	0.150	0.060	0.010	7.374	100.989
	NN	0.300	0.150	0.070	0.010	7.40	87.29
Optimal	GPR	0.300	0.100	0.070	0.019	18.123	30.591
	NN	0.307	0.100	0.070	0.020	18.48	30.15

Table 16: Design parameters of the lowest stress, lowest mass, and optimal compromise solutions from the GPR and NN Pareto fronts.

As shown in Table 16, the optimal compromise designs are near-identical across both models in geometric parameters and objective values, indicating strong surrogate agreement in the interior of the design space. The lowest stress designs converge on the same geometry, with stress values differing by less than 7%. The most notable difference lies in the lowest mass designs, where the NN solution achieves a lower predicted stress despite near-identical mass, and the fillet radius differs slightly between models. This divergence is attributable to the NN being trained on fewer samples (160 versus 300), resulting in sparser coverage of the low-mass boundary region, as is directly visible in the less populated lower-mass end of the NN Pareto front compared to the GPR equivalent.

c) Surrogate validation against FEA

To validate the physical accuracy of both surrogate models, a custom MATLAB script was developed to run the three selected Pareto designs for each surrogate through Abaqus, comparing predicted σ_{max} against the true FEA output and computing the percentage prediction error for each design.

Design	Model	Predicted σ_{max} (MPa)	Abaqus σ_{max} (MPa)	Error (%)
Lowest stress	GPR	24.01	25.7638	6.82
	NN	25.75	25.3427	1.60
Lowest mass	GPR	100.989	102.898	1.85
	NN	87.29	102.3140	14.69
Optimal	GPR	30.591	32.427	5.66
	NN	30.15	30.6661	1.68
Average Error	GPR	4.78%		
	NN	6.00%		

Table 17: Predicted and Abaqus-validated stress values for the lowest stress, lowest mass, and optimal compromise designs from both surrogate models. Obtained from MATLAB file 5.

As shown in Table 17, GPR achieves an average prediction error of 4.78% and NN 6.00%, with both models correctly preserving the ranking of all three designs. The highest errors occur at boundary designs, specifically the lowest mass design for NN (14.69%) and the lowest stress design for GPR (6.82%), where training data density is lowest and both surrogates must extrapolate beyond their training distribution. This is a direct consequence of LHS uniform space-filling, which does not bias sampling towards boundary regions. A boundary-enriched sampling strategy would likely reduce these errors in future work. Notably, despite GPR's superior global RMSE, the NN achieves lower prediction errors at two of the three design points, reinforcing that global RMSE does not guarantee superior accuracy at every individual design point.

4 – Discussion

a) Computational Efficiency

The 300 training samples were generated using the Matlab-Abaqus pipeline at an average FEA simulation time of 27.3 s per sample, totalling 8,190 s of acquisition time. Since the GPR and NN models used 300 and 160 samples respectively, their proportional acquisition costs are computed as:

$$T_{acq,GPR} = 300 \times 27.3 = 8\,190.00 \text{ s}, \quad T_{acq,NN} = 160 \times 27.3 = 4\,368.00 \text{ s}$$

The final training and Pareto generation times were 4.26 s for GPR and 46.31 s for NN. Therefore, the final pipeline costs, including data acquisition, model training, and Pareto front generation, are:

$$T_{final,GPR} = 8\,190 + 4.26 = 8\,194.26 \text{ s}, \quad T_{final,NN} = 4\,368 + 46.31 = 4\,414.31 \text{ s}$$

However, these values only include the final selected models. When all hyperparameter experiments are included, the total model development times become

$$T_{dev,GPR} = 8\,190 + 60.9 = 8\,250.90 \text{ s}, \quad T_{dev,NN} = 4\,368 + 1\,404.4 = 5\,772.40 \text{ s}$$

Although the NN used fewer FEA samples and therefore had a lower acquisition cost, its model selection and optimisation process after data generation was approximately 23 times slower than GPR. This is due to iterative training, early stopping, hyperparameter tuning, and sensitivity to random initialisation. In contrast, GPR required more training data but was much faster and more stable once the dataset had been generated.

If NSGA-II had been conducted directly using Abaqus, the selected optimisation settings of 150 population members and 100 generations would require 15,000 simulations. The estimated runtime would therefore be:

$$T_{FEA} = 15000 \times 27.30 = 409,500 \text{ s} \approx 113.75 \text{ hours}$$

Compared with this direct FEA optimisation, the full surrogate development workflows reduce the total optimisation cost by 50-fold for the GPR model, and by 70-fold for the NN model. The speedup is even greater once the surrogate models have been trained. The final Pareto front generation alone took only 1.612 s for GPR

and 7.087 s for NN, compared with an estimated 409,500 s for direct FEA-based NSGA-II. This demonstrates that both surrogate models make the optimisation computationally practical. Overall, GPR is the stronger model for this problem due to its higher accuracy, greater stability, and much faster model selection, while the NN achieves a lower total proportional pipeline time only because it was trained using fewer FEA samples.

b) Model selection

The GPR model is considered the most appropriate surrogate for this problem, with the NN the least appropriate, based on predictive accuracy, stability, and computational efficiency. GPR achieves a validation RMSE approximately 38% lower and R^2 of 0.9999 compared to 0.9995 for the NN. When validated against Abaqus on selected Pareto designs, GPR achieves an average prediction error of 4.78% against 6.00% for the NN, with both models exhibiting consistent stress underestimation relative to Abaqus, a limitation attributable to sparse boundary sampling as discussed in Section 3c.

The computational argument is strongly in favour of GPR. Although it required a larger training dataset of 300 samples against 160 for the NN, its model selection and optimisation runtime after data generation is approximately 23-fold shorter, demonstrating that the additional data acquisition cost is far outweighed by the efficiency gains in model selection and deployment. Beyond efficiency, GPR is inherently more stable: as demonstrated by the five-seed experiment in Section 2c, NN performance varies substantially across initialisations with an RMSE standard deviation of 0.2736 MPa, requiring multiple training runs to identify a reliable result. GPR produces identical results regardless of initialisation, eliminating this source of uncertainty entirely. Together, GPR delivers superior accuracy, stability, and computational efficiency, consistent with its fundamental suitability for small, low-dimensional datasets with smooth continuous responses, precisely the conditions of this four-variable stress prediction problem.

c) Conditions that favour NN models

While the NN is the least appropriate surrogate in its current form, several changes would shift this conclusion. GPR training complexity scales cubically with dataset size, becoming computationally intractable for higher-fidelity studies requiring thousands of FEA evaluations, whereas NN training scales far more efficiently. Similarly, the four-variable design space considered here is well within the regime where GPR excels, but extending the optimisation to additional geometric or material parameters would degrade GPR performance through the curse of dimensionality, while neural networks are better equipped to learn hierarchical representations across high-dimensional spaces. Response complexity is also relevant: the smooth continuous stress response favours the RBF kernel, but more complex or discontinuous responses such as post-buckling behaviour or progressive material failure would favour the greater flexibility of a neural network. Finally, GPR prediction time grows with dataset size while a trained neural network produces predictions in constant time, making it more scalable for optimisation problems requiring substantially more function evaluations than the 15,000 considered here.

d) Revised constraints

Tightened constraints: tightened bounds would represent a strict subset of the original training domain, meaning both surrogates would remain valid and likely perform with greater accuracy, as all candidate designs would fall within the interior of the training distribution rather than near its boundaries. This is directly supported by Section 3.c, where the lowest prediction errors were observed for the optimal compromise design situated well within the design space interior, while the highest errors corresponded to boundary designs.

Relaxed constraints: relaxed bounds would push the optimisation into regions entirely outside the training distribution where neither surrogate has observed FEA values. Section 3.c already demonstrated errors of up to 6.82% for GPR and 14.69% for NN at the edges of the current bounds, and extrapolation beyond these would likely amplify errors further. The consistent stress underestimation already observed within the training domain suggests this bias would be compounded outside it, raising concerns about the structural safety of designs selected under relaxed constraints.

Improvement strategies: the training dataset should be regenerated using LHS adapted to the new bounds to ensure representative coverage of the updated design space. For relaxed constraints specifically, targeted boundary-enriched sampling would be beneficial, as standard LHS does not bias coverage towards domain

extremes. This would directly address the sparse boundary coverage limitation identified in Section 3.c and improve surrogate reliability across the full relaxed design space.

e) Design improvements

The optimal compromise design from the GPR Pareto front is compared against the average design defined as $W_1 = 0.5$ m, $W_2 = 0.125$ m, $R = 0.05$ m, and $t = 0.015$ m. The compromise design is selected as the most balanced trade-off between safety and mass reduction. Although the NN achieved a lower Abaqus prediction error at the optimal compromise design specifically, GPR demonstrated superior global validation accuracy with an RMSE approximately 37% lower and R^2 of 0.9999 and is therefore considered the more reliable surrogate overall. Since the geometric parameters of both compromise designs are near-identical, the GPR design is selected as the final optimised geometry, with its Abaqus-validated stress later used to confirm physical accuracy.

Design	W_1 (m)	W_2 (m)	R (m)	t (m)	Mass (kg)	VM stress (MPa)
Average	0.500	0.125	0.05	0.015	38.056	44.253
Optimal GPR	0.300	0.100	0.070	0.019	18.123	30.591

Table 18: Comparison of design parameters between the average design and the GPR optimal compromise solution

The optimal GPR compromise design achieves reductions of approximately 30.9% in maximum von Mises stress and 52.4% in mass relative to the average design, confirming that the surrogate-based optimisation delivers substantial simultaneous improvements in structural safety and mass efficiency. A visual comparison of the two panel geometries is provided in the Appendix (1).

Beyond the constraints in Table 1, several practical engineering factors would influence final design selection:

Fatigue life: the framework considers only static von Mises stress, not accounting for cyclic pressurisation throughout service life. Stress concentration around R is particularly relevant for fatigue crack initiation, and minimising peak static stress does not necessarily minimise fatigue damage accumulation.

Buckling resistance: a thinner, wider panel may satisfy the static von Mises criterion while remaining susceptible to out-of-plane buckling under compressive or shear loads.

Damage modelling: linear elastic material behaviour is assumed, meaning progressive failure mechanisms such as yielding and crack propagation are not represented.

Manufacturing constraints: standard aluminium sheet thicknesses are available only in discrete increments, meaning continuously optimised thickness values may not correspond to available stock sizes.

Material extension: extending the design to composites such as CFRP would introduce fibre orientation and laminate stacking sequence as additional design variables beyond what the von Mises criterion captures.

Each consideration could be incorporated into future surrogate models by extracting additional FEA outputs such as buckling load and fatigue life as extra objectives, enforcing discrete manufacturing constraints as penalty functions, and expanding input dimensionality to include composite design variables, which as discussed in Section 4(c) would favour the NN over GPR.

5 – Conclusion

The surrogate-based multi-objective optimisation framework successfully demonstrated the feasibility of replacing FEA evaluations with data-driven surrogate models for the shape optimisation of an aircraft maintenance panel. GPR demonstrated superior predictive accuracy and computational efficiency over the NN, achieving a validation RMSE approximately 38% lower at a fraction of the training time. The optimal compromise design achieved reductions of approximately 30.9% in maximum von Mises stress and 52.4% in mass relative to the average design, at a computational speedup of approximately 50-fold over direct FEA optimisation. Key limitations include sparse boundary sampling contributing to Abaqus validation errors of up to 14%, and the restriction to static stress objectives under linear elastic assumptions. Future work should address these through boundary-enriched sampling strategies and the incorporation of additional objectives such as fatigue life and buckling resistance.

6 – Appendix

1) Average vs optimised design

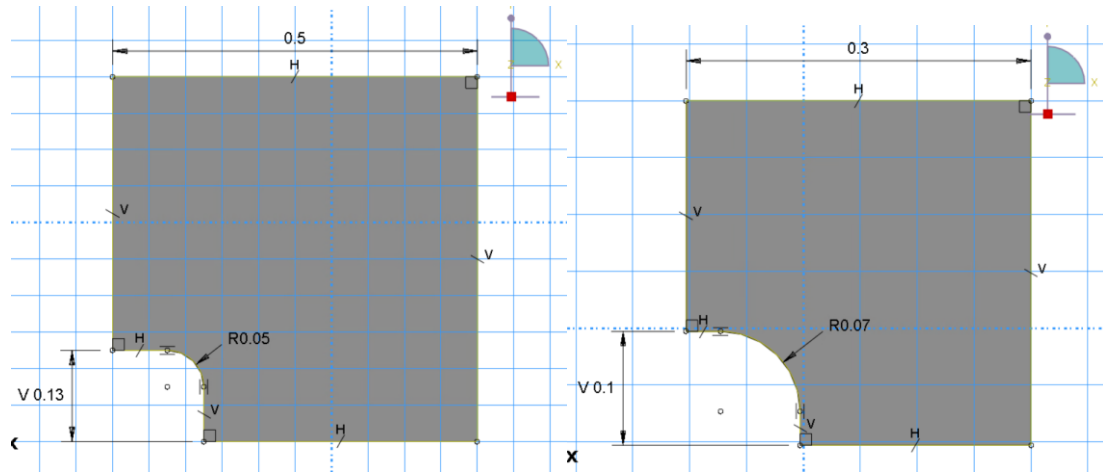


Figure 6, 7: Geometry comparison between the average design and the optimised solution

7 – Acknowledgement and references

ChatGPT (version 5.3, OpenAI, chatgpt.com) was used in a limited assistive role during the preparation of this report. The tool was used to review grammar and syntax in the written text. ChatGPT was not used to generate analysis methods, code implementations, or ideas related to the coursework. All analysis methods, modelling choices, and code were developed independently or with the help of MECH0107 class content.

- [1] Vardhan, H. and Sztipanovits, J. (2022). *Deep Learning-based Finite Element Analysis (FEA) surrogate for sub-sea pressure vessel*. [online] arXiv.org. Available at: <https://arxiv.org/abs/2206.03322> [Accessed 7 Apr. 2026].
- [2] Almutairi, M.A.M., Dunning, P. and Maheri, A. (2026). Multiobjective optimization of UAV winglet performance using surrogate models and CFD. *Engineering with Computers*, 42(1). doi:<https://doi.org/10.1007/s00366-025-02249-2>.
- [3] docs.scipy.org. (n.d.). *scipy.stats.qmc.LatinHypercube — SciPy v1.10.1 Manual*. [online] Available at: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.qmc.LatinHypercube.html>.
- [4] Rasmussen, C. and Williams, C. (2006). *Gaussian Processes for Machine Learning*. [online] Available at: <https://gaussianprocess.org/gpml/chapters/RW.pdf>.
- [5] Gholamy, A., Kreinovich, V. and Kosheleva, O. (2018). Why 70/30 or 80/20 Relation Between Training and Testing Sets: A Pedagogical Explanation. *Departmental Technical Reports (CS)*. [online] Available at: https://scholarworks.utep.edu/cs_techrep/1209/.
- [6] Shi, X., Jiang, D., Qian, W. and Liang, Y. (2022). Application of the Gaussian Process Regression Method Based on a Combined Kernel Function in Engine Performance Prediction. *ACS Omega*, 7(45), pp.41732–41743. doi:<https://doi.org/10.1021/acsomega.2c05952>.
- [7] Scikit-learn.org. (2015). 1.7. *Gaussian Processes — scikit-learn 0.21.3 documentation*. [online] Available at: https://scikit-learn.org/stable/modules/gaussian_process.html.
- [8] Raschka, S. (2018). *Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning*. [online] arXiv.org. Available at: <https://arxiv.org/abs/1811.12808>.

[9] Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. [online] Deeplearningbook.org. Available at: <https://www.deeplearningbook.org/contents/regularization.html>.